# Auth & User Management API reference

Version 1.0 (draft for review) · 15 Oct 2026 · Shubham Sinha

The module exposes 18 REST endpoints under `/api/v1`. It covers the nine SRS items under User Authentication & User Management: patient registration by a doctor, doctor registration by an admin, admin registration, secure login, forgot password, email verification, role-based access control, basic profile and medical profile.

It also owns the `users` collection. Other modules use it to find out who is calling, what role they have, which doctors a patient is linked to, and which time zone a patient is in. Health Monitoring already reads this collection.

> **Database:** this module uses MongoDB, the same database as Health Monitoring. The SRS lists PostgreSQL/MySQL, so the team still has to confirm this choice for the whole project (see Open questions).

## Conventions

These are the same conventions as the Health Monitoring API.

| Item | Rule |
|---|---|
| Base URL | `/api/v1` |
| Auth header | `Authorization: Bearer <Firebase ID token>` on every route |
| Request ID | Optional `X-Request-Id` header. The server creates one if missing, returns it on every response and stores it in audit records |
| Times | ISO 8601 with offset, for example `2026-10-15T10:30:00+05:30` |
| Dates | `YYYY-MM-DD` |
| IDs | MongoDB ObjectId strings. This is the user's id in our database, not their Firebase uid |
| Pagination | `?limit=20&cursor=<nextCursor>`. Lists return `{ data, nextCursor }`. Maximum limit is 100 |
| Errors | One format with a stable machine-readable `code`. See Errors |
| Docs | Swagger UI at `/api/docs` |

## Who does what: Firebase and this API

Firebase Authentication handles passwords. This API never sees or stores a password.

| SRS item | Handled by | How |
|---|---|---|
| Secure Login | Firebase (in the app) | The app signs in with Firebase email and password and gets an ID token. It sends that token on every API call. Tokens expire after 1 hour, and the Firebase SDK refreshes them automatically |

| SRS item | Handled by | How |
|---|---|---|
| Forgot Password | Firebase (in the app) | The app calls Firebase `sendPasswordResetEmail`. Firebase emails a reset link. No API route is needed |
| Email Verification | Firebase and this API | New users set their first password through an emailed link, which proves they own the email address. `GET /auth/me` returns `emailVerified`, and other routes refuse unverified users. See Onboarding |
| Patient / Doctor / Admin Registration | This API | `POST /patients`, `POST /doctors`, `POST /admins`. Each one creates the Firebase account and the profile together |
| Role-Based Access Control | This API | The role is stored in `users.role` and checked on every route. A role in the request body or in the app is never trusted |
| Basic / Medical Profile | This API | `/users/me/profile`, `/patients/…/medical-profile` |

# Endpoints

| Method | Route | Role | Purpose |
|---|---|---|---|
| **Identity and own profile** | | | |
| GET | `/auth/me` | Any | Who am I: id, role, name, email verified. The app calls this after login to choose the patient, doctor or admin screens |
| GET | `/users/me/profile` | Any | My basic profile |
| PATCH | `/users/me/profile` | Any | Update my basic profile |
| **Registration and user management** | | | |
| POST | `/admins` | Admin | Register another admin |
| POST | `/doctors` | Admin | Register a doctor |
| GET | `/doctors` | Admin | List doctors |
| GET | `/doctors/:doctorId` | Admin | One doctor |
| PATCH | `/doctors/:doctorId` | Admin | Update a doctor, or deactivate or reactivate them |
| POST | `/patients` | Doctor, Admin | Register a patient |
| GET | `/patients` | Doctor, Admin | Patient list. A doctor sees only their linked patients; an admin sees all |
| GET | `/patients/:patientId` | Doctor, Admin | One patient |
| PATCH | `/patients/:patientId` | Doctor, Admin | Update a patient's basic details. Only an admin can deactivate or reactivate |
| PUT | `/patients/:patientId/doctors` | Admin | Set which doctors a patient is linked to |

| Method | Route | Role | Purpose |
|---|---|---|---|
| POST | `/users/:userId/invitations` | Admin, Doctor | Send the set-password email again |
| **Medical profile** | | | |
| GET | `/patients/me/medical-profile` | Patient | My medical profile |
| PATCH | `/patients/me/medical-profile` | Patient | Update my medical profile |
| GET | `/patients/:patientId/medical-profile` | Doctor | A linked patient's medical profile |
| PATCH | `/patients/:patientId/medical-profile` | Doctor | Update a linked patient's medical profile |

There is no DELETE route. Accounts are deactivated, never deleted, so a patient's health history and audit trail stay intact (see Account lifecycle).

## Access rules

| Rule | Detail |
|---|---|
| Identity | The caller is found from the verified Firebase token ( `uid` → `users.firebaseUid` ). A valid token with no active profile gets `401` . `/me` routes never take a user id from the request |
| Email verified | Every route except `GET /auth/me` returns `403 EMAIL_NOT_VERIFIED` until the caller's email is verified |
| Doctor → patient | A doctor can reach only patients whose `doctorIds` include them. Any other patient returns `404` , so a doctor can't tell whether that patient exists. This is the same rule as Health Monitoring |
| Admin → medical data | Admins manage accounts but cannot read medical profiles or health data. They have no clinical need for it |
| Patient → others | A patient can reach only their own profile and medical profile |
| Deactivated users | Their Firebase account is disabled and their sessions are revoked, so they can't log in and existing tokens stop working |

## Onboarding: registration and first login

1. A doctor (for a patient) or an admin (for a doctor or admin) calls the matching `POST` route with the person's name and email.
2. The API creates the Firebase account with a **random password that nobody knows** and with `emailVerified: false` , then saves the profile in `users` . If the second step fails, the Firebase account is removed, so nothing is half-created.
3. Firebase emails the new user a link to set their password. The API sends this through Firebase's own email service, so no extra mail server is needed for version 1.
4. The user opens the link, chooses a password, and logs in to the app. Setting a password through the emailed link proves they own the address, so their email counts as verified.

5. The app calls `GET /auth/me`. If `emailVerified` is still `false`, the app shows a "verify your email" screen and can ask Firebase to send a verification email (`sendEmailVerification`).

If the link expires or the email was lost, a doctor or admin calls `POST /users/:userId/invitations` to send it again.

**The first admin** can't be registered through the API, because no admin exists yet to call it. It is created once from the server with a command-line script (`npm run create-admin`). Every later admin is registered with `POST /admins`.

# GET /auth/me

Works even while the email is unverified, so the app can show the right screen. Response `200`:

```
{ "id": "6701f0a9e4b0f1a2b3c4d001", "role": "PATIENT", "name": "Rajesh Iyer",
  "email": "rajesh@example.com", "emailVerified": true, "active": true }
```

Errors: `401` when the token is missing, invalid or expired, or when no active profile exists for it.

# Basic profile

### GET /users/me/profile

Response `200`. The fields depend on the role:

```
{ "id": "6701f0a9e4b0f1a2b3c4d001", "role": "PATIENT", "name": "Rajesh Iyer",
  "email": "rajesh@example.com", "phone": "+919876543210",
  "timeZone": "Asia/Kolkata",
  "doctors": [ { "id": "6700aa01e4b0f1a2b3c4d500", "name": "Dr Meera Nair" } ],
  "createdAt": "2026-10-15T10:30:00+05:30", "updatedAt": "2026-10-15T10:30:00+05:30" }
```

- Patients: `timeZone` and `doctors` (id and name only).
- Doctors: `specialization`.
- Admins: neither.

### PATCH /users/me/profile

Send only the fields you want to change.

| Field | Who | Rule |
|---|---|---|
| `name` | Any | 1–100 characters |
| `phone` | Any | E.164 format, for example `+919876543210`, or `null` to clear |
| `timeZone` | Patient | A valid IANA zone, for example `Asia/Kolkata`. As Health Monitoring v1.1 states, a change affects future days only |
| `specialization` | Doctor | Up to 100 characters, or `null` |

These fields cannot be changed here: `email` (see Open questions), `role`, `active` and the doctor links. Sending any of them returns `400`.

Response `200`: the updated profile. Errors: `400`, `401`, `403`.

# Registration

### POST /patients

```
{ "name": "Rajesh Iyer", "email": "rajesh@example.com", "phone": "+919876543210",
  "timeZone": "Asia/Kolkata", "doctorIds": ["6700aa01e4b0f1a2b3c4d500"] }
```

- `name` and `email` are required. `phone` is optional. `timeZone` is optional and defaults to `Asia/Kolkata`.
- **Doctor caller:** the patient is linked to that doctor automatically, and sending `doctorIds` returns `400`.
- **Admin caller:** `doctorIds` is required, with at least one active doctor.
- The email is stored in lower case and must not belong to any existing user.

Response `201`: the patient, as in `GET /patients/:patientId`, with `"invitation": "SENT"`. If the email couldn't be sent, `invitation` is `"FAILED"`. The patient is still registered, and the email can be sent again with `POST /users/:userId/invitations`. The same applies to `POST /doctors` and `POST /admins`.

Errors: `400`, `403`, `404` unknown doctor, `409 EMAIL_ALREADY_EXISTS`.

### POST /doctors

Same flow, admin only. Body: `{ "name", "email", "phone"?, "specialization"? }`. Response `201`: the doctor. Errors: `400`, `403`, `409 EMAIL_ALREADY_EXISTS`.

### POST /admins

Admin only. Body: `{ "name", "email", "phone"? }`. Response `201`. Errors: `400`, `403`, `409 EMAIL_ALREADY_EXISTS`.

### POST /users/:userId/invitations

Sends the set-password email again. There is no request body.

- An admin can do this for any user.
- A doctor can do it only for their linked patients. Any other user returns `404`.
- It also works after the user has set a password: the email then acts as a password reset.

Response `202`: `{ "invitation": "SENT" }`, or `"FAILED"` if Firebase could not send the email. Errors: `404`, `409 USER_INACTIVE`, and `429` if called more than 5 times per user per hour.

# Patient and doctor management

### GET /patients

| Query parameter | Default | Notes |
|---|---|---|
| `q` | | Search by name or email (contains, case-insensitive), 2–100 characters |
| `active` | `true` | `true`, `false` or `all` |
| `doctorId` | | Admin only: patients linked to this doctor |
| `limit`, `cursor` | 20 | Sorted by name |

Response `200`:

```
{ "data": [
  { "id": "6701f0a9e4b0f1a2b3c4d001", "name": "Rajesh Iyer", "email": "rajesh@example.com",
    "phone": "+919876543210", "timeZone": "Asia/Kolkata", "active": true, "emailVerified": true,
    "doctors": [ { "id": "6700aa01e4b0f1a2b3c4d500", "name": "Dr Meera Nair" } ],
    "createdAt": "2026-10-15T10:30:00+05:30" }
], "nextCursor": null }
```

`GET /patients/:patientId` returns one entry in the same shape. It returns `404` for an unknown patient, or for a patient not linked to the calling doctor.

### PATCH /patients/:patientId

- Doctor (linked) and admin: `name`, `phone`, `timeZone`, with the same rules as the basic profile.
- Admin only: `active` (`true` or `false`). See Account lifecycle.

Response `200`: the patient. Errors: `400`, `403` when a doctor sends `active`, `404`.

### PUT /patients/:patientId/doctors

Admin only. It replaces the whole list: `{ "doctorIds": ["…", "…"] }`, with 1–10 active doctors and no duplicates. A doctor removed from the list loses access to that patient immediately, including their vitals and alerts in Health Monitoring.

Response `200`: the patient. Errors: `400`, `404` unknown patient or doctor.

### GET /doctors, GET /doctors/:doctorId, PATCH /doctors/:doctorId

Admin only. These work like the patient routes. Query parameters are `q`, `active`, `limit` and `cursor`. Each doctor includes `specialization` and `patientCount` (active linked patients).

PATCH accepts `name`, `phone`, `specialization` and `active`. A doctor who is the **only** doctor of any active patient cannot be deactivated: the request returns `409 DOCTOR_HAS_PATIENTS`, and the admin must relink those patients first.

## Medical profile

There is one medical profile per patient, stored apart from the basic profile so that admins never load it. The patient and their linked doctors can read and edit it, and every change is audited with the old and new value.

> **The fields below are a proposal.** The SRS lists "Medical Profile Management" but not its fields. It also says the client provides the patient registration form. Following SRS section 14, the clinical team must confirm these fields before release (see Open questions).

| Field | Type | Rule |
|---|---|---|
| `dateOfBirth` | date | In the past, age 0–120 |
| `sex` | enum | `MALE`, `FEMALE`, `OTHER`, `UNDISCLOSED` |

| Field | Type | Rule |
|---|---|---|
| `heightCm` | number | 50–250. Weight is not stored here; it is logged daily in Health Monitoring |
| `bloodGroup` | enum | `A+`, `A-`, `B+`, `B-`, `AB+`, `AB-`, `O+`, `O-`, `UNKNOWN` |
| `diagnosis` | text | Up to 500 characters, for example the cancer type |
| `plannedSurgery` | text | Up to 200 characters |
| `plannedSurgeryDate` | date | Any date. It may be changed when surgery is rescheduled |
| `comorbidities` | list of text | Up to 20 items of 100 characters each |
| `allergies` | list of text | Up to 20 items of 100 characters each |
| `medications` | list of text | Up to 30 items of 200 characters each |
| `smokingStatus` | enum | `NEVER`, `FORMER`, `CURRENT` |
| `emergencyContact` | object | `{ name, relation, phone }`, phone in E.164 |

**GET**: response `200` contains every field (`null` when empty), plus `version`, `updatedAt` and `updatedBy: { id, role }`. If nothing has been saved yet, every field is `null` and `version` is `0`.

**PATCH**: send only the fields that change, plus the `version` you last read. `null` clears a field. If someone else saved first, the request returns `409 CONCURRENT_UPDATE`; reload and try again. The response is the updated profile.

Errors: `400`, `401`, `403`, `404` (doctor routes: unknown or unlinked patient), `409 CONCURRENT_UPDATE`.

## Account lifecycle

| Event | What happens |
|---|---|
| Registered | Firebase account and profile are created together, and the set-password email is sent |
| Deactivated (admin) | `active: false`. The Firebase account is disabled and its sessions revoked. Every API, including Health Monitoring, returns `401`. Data is kept |
| Reactivated (admin) | `active: true`. The Firebase account is enabled again, and the user logs in with their old password |
| Deleted | Never in version 1. Medical records have to be kept. Rules for removing data on request are an open question |

## Data model (MongoDB)

| Collection | Fields |
|---|---|
| `users` | `firebaseUid` (unique), `role`, `name`, `email` (unique, lower case), `phone`, `active`, `timeZone` and `doctorIds` (patients), `specialization` (doctors), `createdBy`, `createdAt`, `updatedAt`. This keeps every field Health Monitoring already reads, unchanged |
| `medicalProfiles` | `patientId` (unique), the fields above, `version`, `updatedBy`, `updatedAt` |

| Collection | Fields |
| --- | --- |
| `auditLogs` | The same shape and collection as Health Monitoring |

## Audit log

These records use the same format as Health Monitoring. Change records are written in the same transaction as the change.

| action | Recorded when |
| --- | --- |
| `USER_REGISTERED` | A patient, doctor or admin is created |
| `PROFILE_UPDATED` | Basic profile fields change (before and after) |
| `USER_DEACTIVATED` / `USER_REACTIVATED` | An admin changes `active` |
| `DOCTOR_LINKS_CHANGED` | A patient's doctors change (before and after) |
| `INVITATION_SENT` | A set-password email is sent |
| `MEDICAL_PROFILE_VIEWED` | A doctor reads a patient's medical profile. If this can't be written, the call fails with 500 |
| `MEDICAL_PROFILE_UPDATED` | Any change (before and after) |

## Rate limits

| Scope | Limit |
| --- | --- |
| `POST /patients`, `/doctors`, `/admins` | 20 requests per minute per user |
| `POST /users/:userId/invitations` | 5 per target user per hour |
| All other routes | 120 requests per minute per user |
| Any route, by IP address | 300 requests per minute |

A `429` response includes a `Retry-After` header. Login attempts are rate-limited by Firebase.

## Errors

The format is the same as Health Monitoring:

```
{ "statusCode": 409, "code": "EMAIL_ALREADY_EXISTS", "error": "Conflict",
  "message": "A user with this email already exists", "requestId": "b1c2…" }
```

| Status | code | When |
|---|---|---|
| 400 | `VALIDATION_FAILED` | The body or query fails validation. `details[]` names each field, and `details[].code` is one of `REQUIRED`, `WRONG_SHAPE`, `OUT_OF_RANGE`, `TOO_LONG` or `NOT_ALLOWED` (a field this caller's role may not send) |
| 401 | `UNAUTHENTICATED` | The token is missing, invalid or expired, or the account is inactive |
| 403 | `FORBIDDEN_ROLE` | The caller's role can't use this route or field |
| 403 | `EMAIL_NOT_VERIFIED` | The caller's email is not verified yet |
| 404 | `NOT_FOUND` | The record doesn't exist, or the caller can't access it |
| 409 | `EMAIL_ALREADY_EXISTS` | The email is already used by another user |
| 409 | `DOCTOR_HAS_PATIENTS` | Deactivation would leave an active patient with no doctor |
| 409 | `USER_INACTIVE` | An invitation was requested for a deactivated user |
| 409 | `CONCURRENT_UPDATE` | Someone else changed the record first. Nothing was saved; reload and retry |
| 429 | `RATE_LIMITED` | The rate limit was exceeded |
| 500 | `INTERNAL` | Unexpected server error. Details are logged, never returned |

# Open questions

| Question | Owner |
|---|---|
| Database for the whole project: MongoDB (as built) or PostgreSQL/MySQL (as the SRS says)? | Team, client |
| Medical profile fields, and whether patients may edit them or only doctors | Clinical team |
| Fields on the patient registration form (SRS section 7 lists it as a client input) | Client |
| Doctor details to store, for example registration number or hospital | Client |
| Can a user change their email? (Not in version 1; an admin would have to re-register them) | Team |
| Data deletion on request, and how long records are kept | Client, legal |
| Branded emails: Firebase's default templates for now, or the client's SMTP service later (SRS section 7) | Client |

# Not in this module

- Consent forms and privacy policy acceptance. That is the SRS Consent & Privacy module, built separately.
- Exercise plans, notifications and dashboards.
- Social login, phone or OTP login, and two-factor authentication. The SRS doesn't list them.